

UNIVERSIDAD DE SALAMANCA

GRADO EN INGENIERIA INFORMATICA

ANEXO VIII

Uso de Inteligencia Artificial

Sistema de Gestion Documental mediante Blockchain



David Pérez Velasco

Tutores:

Gabriel Villarrubia González

Sergio García González

Julio 2026

Trabajo de Fin de Grado

Índice

1. Declaracion de uso de IA	3
2. Herramientas utilizadas	3
3. Ejemplos de uso	3
3.1. Depuracion de errores de sincronizacion blockchain	3
3.2. Optimizacion de busqueda documental	4
3.3. Generacion de diagramas de secuencia BCE	4
3.4. Documentacion tecnica del sistema de cifrado	4
3.5. Diseno del esquema de base de datos	5
3.6. Implementacion del flujo de transferencia de propiedad	5
3.7. Configuracion del pipeline CI/CD	5
3.8. Generacion de la matriz de rastreabilidad	6
3.9. Diseno de la interfaz de verificacion publica	6
3.10. Refactorizacion del sistema de notificaciones	6
4. Registro de uso por area del proyecto	7
5. Limitaciones y lecciones aprendidas	7
6. Garantia de originalidad	8

1. Declaracion de uso de IA

En este apartado se detalla el uso que se ha dado a la inteligencia artificial durante el desarrollo del proyecto.

El autor declara que, en el desarrollo del presente Trabajo de Fin de Grado, se ha empleado la Inteligencia Artificial exclusivamente como herramienta de asistencia tecnica. En ningun momento el sistema ha sido codificado de forma autonoma por un modelo de lenguaje, manteniendose en todo momento el control humano sobre el desarrollo integral del proyecto.

El uso de la IA se ha limitado a las siguientes tareas de soporte:

- Depuracion de errores y resolucion de problemas tecnicos.
- Optimizacion de fragmentos de codigo especificos.
- Generacion de documentacion tecnica y diagramas UML.
- Asistencia en la redaccion de la memoria y anexos.

Todas las sugerencias generadas por las herramientas de IA han sido supervisadas, verificadas y adaptadas manualmente para garantizar su correccion tecnica y su ajuste a los requisitos del proyecto.

2. Herramientas utilizadas

A continuacion se enumeran las herramientas de IA empleadas durante el desarrollo, indicando el modelo y las tareas en las que prestaron asistencia.

Herramienta	Version/Modelo	Tareas realizadas
ChatGPT	GPT-4o	Depuracion de errores, asistencia en arquitectura, redaccion de documentacion
GitHub Copilot	Copilot Chat	Sugerencias de codigo, autocompletado, refactorizacion
Claude	Claude 3.5 Sonnet	Revision de logica compleja, optimizacion de queries, generacion de diagramas

Cuadro 1: Herramientas de IA empleadas en el proyecto

3. Ejemplos de uso

A continuacion se presentan ejemplos representativos de los prompts utilizados y las tareas realizadas con asistencia de IA. Se incluye tanto el contexto de la peticion como el resultado obtenido y las adaptaciones manuales posteriores.

3.1. Depuracion de errores de sincronizacion blockchain

Contexto: Durante la implementacion del `Event Listener`, se detecto que las transacciones confirmadas en la blockchain no se reflejaban en PostgreSQL. El evento se emitia correctamente desde el smart contract, pero la actualizacion en la capa de persistencia fallaba silenciosamente.

Prompt: *“El servicio de escucha de eventos blockchain no sincroniza correctamente el estado de la base de datos tras una transaccion confirmada. El evento se emite pero la actualizacion en*

la capa de persistencia falla silenciosamente. El código utiliza `ethers.js v6` y un listener basado en `contract.on(eventName, callback)`. El callback procesa el evento y llama a Prisma para actualizar PostgreSQL, pero nunca se ejecuta la actualización.”

Resultado: Se identificó que el problema radicaba en la falta de manejo de excepciones en el callback del event listener. La IA sugirió implementar un bloque try/catch con reintentos exponenciales, un sistema de cola de eventos pendientes y un mecanismo de *checkpoint* para evitar el reprocesamiento de eventos ya sincronizados. Tras la revisión manual, se implementó la solución en `blockchainSync.ts`, mejorando adicionalmente el sistema de logs para trazabilidad.

3.2. Optimización de búsqueda documental

Contexto: La búsqueda de documentos del usuario se realizaba filtrando arrays en memoria con JavaScript, lo que provocaba tiempos de respuesta inaceptables con más de 10,000 registros en la base de datos.

Prompt: “Optimiza la función de búsqueda de documentos que actualmente realiza un filtrado en memoria con arrays de JavaScript. La base de datos tiene más de 10,000 registros y el rendimiento es inaceptable. Se necesita mantener la paginación y el filtrado por múltiples criterios (nombre, tags, tipo, fecha). El backend utiliza Express con Prisma ORM sobre PostgreSQL.”

Resultado: Se refactorizó la lógica para delegar el filtrado al motor de base de datos mediante consultas parametrizadas con `Prisma.where`, índices compuestos en PostgreSQL sobre los campos de búsqueda, y paginación basada en cursores. El tiempo de respuesta pasó de 2.3 segundos a 120 ms en la carga inicial del listado.

3.3. Generación de diagramas de secuencia BCE

Contexto: Para el Anexo III (Análisis y Diseño del Sistema) se requería generar 39 diagramas de secuencia de análisis (BCE) a partir de las especificaciones de casos de uso ya documentadas en el Anexo I.

Prompt: “Genera un diagrama de secuencia PlantUML para el caso de uso ‘Firmar documento con wallet blockchain’, incluyendo las interacciones con el smart contract. Utiliza estereotipos Boundary, Control y Entity. El diagrama debe seguir el estilo definido en `_sequence-bce-style.puml`. El flujo es: usuario selecciona documento, solicita firma, conecta wallet, firma challenge, backend confirma en blockchain.”

Resultado: Se generó un diagrama de secuencia base que fue posteriormente adaptado manualmente para reflejar la lógica real del sistema: gestión de wallets EIP-6963 y WalletConnect, challenge-firma con recuperación de dirección vía `ethers.verifyMessage()`, y confirmación on-chain con el patrón prepare/confirm. La IA aceleró significativamente el proceso de creación de la estructura básica de los 39 diagramas, que luego fueron revisados y corregidos uno a uno.

3.4. Documentación técnica del sistema de cifrado

Contexto: Era necesario documentar la arquitectura de cifrado de tres capas del sistema para el Anexo IV (Documentación Técnica), incluyendo los algoritmos, los flujos de clave y las decisiones de diseño.

Prompt: “Explica la arquitectura de cifrado del sistema DocumentChain, incluyendo AES-256-GCM para cifrado de documentos, ECDH P-256 para intercambio de claves entre usuarios al compartir documentos, Argon2id para derivación de contraseñas, y el patrón prepare/confirm para transacciones blockchain. La explicación debe ser técnica pero accesible para un tribunal de ingeniería informática.”

Resultado: Se obtuvo una explicación estructurada que sirvió como base para la redacción del Anexo IV, posteriormente verificada y ampliada con los detalles de implementación reales,

incluyendo el pseudocódigo de cifrado, el diagrama de flujo de claves y las referencias a los módulos concretos del código fuente (`FileCrypto.ts`, `KeyManager.ts`).

3.5. Diseño del esquema de base de datos

Contexto: Durante la fase de diseño, se necesitó iterar sobre el esquema Prisma para incorporar el modelo de wallets múltiples, notificaciones y preferencias de usuario, manteniendo la integridad referencial y el rendimiento.

Prompt: *“Revisa este esquema Prisma para una aplicación de gestión documental con blockchain. Se necesita añadir soporte para múltiples wallets por usuario (Ethereum), un sistema de notificaciones con preferencias por tipo, y un modelo de sesiones con refresh tokens. El esquema actual tiene User, Document, Version, Signature, Share y Event. Sugiere las relaciones, índices necesarios y restricciones.”*

Resultado: La IA propuso un modelo de datos con las tablas `Wallet` (relación 1:N con `User`), `Notification` y `NotificationPreference`, y `Session` con soporte para refresh tokens rotativos. Las sugerencias fueron revisadas, ajustadas (se eliminó el modelo de organización por carpetas en BD al considerarse innecesario para el alcance del proyecto) e implementadas mediante migraciones de Prisma.

3.6. Implementación del flujo de transferencia de propiedad

Contexto: La transferencia de propiedad de un documento requería un flujo complejo que involucraba re-cifrado de la clave simétrica para el nuevo propietario, registro on-chain del cambio de titularidad, y notificación a ambas partes.

Prompt: *“Implementa el servicio de transferencia de propiedad de un documento en TypeScript. El flujo debe: 1) verificar que el solicitante es el propietario actual, 2) descifrar la clave simétrica con la clave privada del propietario actual, 3) recifrar la clave para el nuevo propietario usando su clave pública (ECDH), 4) preparar la transacción blockchain de cambio de titularidad, 5) confirmar la transacción tras la firma del usuario, 6) notificar a ambas partes. Usa Prisma para la persistencia y ethers.js v6 para blockchain.”*

Resultado: Se generó la estructura completa del `TransferService` con los métodos `prepareTransfer` y `confirmTransfer`. La revisión manual ajustó el manejo de errores para casos límite (documento archivado, nuevo propietario sin clave pública, transferencia de documento público sin re-cifrado) y se implementaron los tests unitarios correspondientes en `transferService.test.ts`.

3.7. Configuración del pipeline CI/CD

Contexto: Se necesitaba configurar un pipeline de integración continua con GitHub Actions para ejecutar tests, construir las imágenes Docker y desplegar en un servidor autoalojado.

Prompt: *“Configura un workflow de GitHub Actions para un proyecto monorepo con frontend (React/Vite), backend (Node.js/Express) y smart contracts (Hardhat). El pipeline debe: ejecutar tests unitarios en los tres módulos, construir la imagen Docker del backend, construir los assets del frontend, y notificar el resultado. El despliegue se realiza en un runner autoalojado con Docker Compose.”*

Resultado: Se obtuvo una configuración base de `ci.yml` que fue adaptada para incluir la generación de artefactos, la construcción condicional solo en la rama principal, el uso de secretos de repositorio para variables de entorno, y la notificación de estado mediante webhooks. El workflow resultante se documentó en el Anexo VII (Manual de Montaje).

3.8. Generacion de la matriz de rastreabilidad

Contexto: La matriz de rastreabilidad del Anexo I requería cruzar 43 casos de uso con requisitos funcionales (7 IRQ), no funcionales (6 NFR) y diagramas de secuencia (86 diagramas entre BCE y diseño). La generación manual de estas tablas era propensa a errores y consumía mucho tiempo.

Prompt: “Genera el código LaTeX para una matriz de rastreabilidad que cruce los casos de uso UC-0001 a UC-0039 con los requisitos de información IRQ-0001 a IRQ-0007 y los requisitos no funcionales NFR-0001 a NFR-0006. La matriz debe usar los datos de las tablas de especificación de casos de uso. Formato: tabla con filas por UC, columnas por IRQ/NFR, celdas con X donde exista relación.”

Resultado: La IA generó las tablas de rastreabilidad con la estructura LaTeX correcta, que fueron posteriormente verificadas manualmente contra las especificaciones de cada caso de uso. Se detectaron y corrigieron 3 omisiones en el cruce UC-IRQ y se añadieron las referencias a los diagramas de secuencia correspondientes.

3.9. Diseño de la interfaz de verificación pública

Contexto: La página de verificación pública de documentos debía permitir a cualquier usuario (sin necesidad de registro) verificar la autenticidad de un documento mediante su hash y consultar su trazabilidad en blockchain.

Prompt: “Diseña un componente React para la verificación pública de documentos en blockchain. La página debe: tener un campo de entrada para el hash del documento, mostrar los metadatos del documento si existe en la blockchain, mostrar el historial de versiones y firmas, e indicar claramente si el documento es auténtico o no. Utiliza Tailwind CSS para el estilo y componentes shaden/ui. La página debe ser responsive.”

Resultado: Se generó la estructura base del componente `PublicVerifyPage` con los estados de carga, éxito y error. La revisión manual ajustó la integración con el hook `useBlockchainQueries`, añadió el soporte para múltiples wallets en la verificación de firmas, y mejoró la experiencia de usuario con indicadores visuales de estado (verde/rojo) para la autenticidad.

3.10. Refactorización del sistema de notificaciones

Contexto: El sistema de notificaciones existente era sincrónico y bloqueaba las peticiones HTTP mientras esperaba la confirmación del servidor SMTP. Era necesario migrar a un modelo asíncrono con cola de trabajos.

Prompt: “Refactoriza el servicio de notificaciones por correo electrónico para que sea asíncrono. Actualmente `EmailService.send()` es una función `async` que bloquea la respuesta HTTP hasta que Postfix confirma el envío. Se necesita que las notificaciones se encolen y se procesen en segundo plano, sin afectar al tiempo de respuesta de la API. El backend usa Express y Nodemailer.”

Resultado: La IA sugirió implementar un patrón de cola en memoria con procesamiento periódico mediante `node-cron`, almacenando las notificaciones pendientes en la tabla `Notification` de PostgreSQL. La solución fue adaptada para incluir un sistema de reintentos con backoff exponencial (3 intentos con intervalos de 1, 5 y 15 minutos) y notificación de fallo persistente al administrador. El cambio redujo el tiempo medio de respuesta de los endpoints afectados en un 40 %.

4. Registro de uso por area del proyecto

En este apartado se resume la intensidad de uso de IA en cada area del proyecto, permitiendo al tribunal valorar el alcance real de la asistencia automatizada.

La tabla 2 resume la intensidad de uso de IA en cada area del proyecto, permitiendo al tribunal valorar el alcance real de la asistencia automatizada.

Area del proyecto	Intensidad IA	Descripcion del uso
Arquitectura del sistema	Baja	Decisiones de disenio tomadas manualmente. IA consultada para validar alternativas.
Smart contracts (Solidity)	Media	Asistencia en la implementacion de patrones (Access-Control, ReentrancyGuard) y generacion de NatSpec.
Backend (Node.js/Express)	Media	Depuracion de errores, optimizacion de consultas Prisma y refactorizacion de servicios.
Frontend (React/Vite)	Media	Generacion de componentes shadcn/ui, asistencia en gestion de estado con Zustand.
Cifrado (FileCrypto, KeyManager)	Baja	Implementacion manual basada en WebCrypto y estandares NIST. IA consultada para verificacion.
Pruebas unitarias y E2E	Media	Generacion de casos de prueba iniciales, revision manual y ampliacion de cobertura.
Diagramas UML (PlantUML)	Alta	Generacion de la estructura base de los 78 diagramas de secuencia. Revision y correccion manual de cada uno.
Documentacion LaTeX	Alta	Asistencia en la redaccion de todos los anexos y la memoria. Revision y adaptacion manual del contenido.
Despliegue (Docker, CI/CD)	Media	Configuracion inicial de Docker Compose y GitHub Actions, ajustada manualmente.

Cuadro 2: Intensidad de uso de IA por area del proyecto

5. Limitaciones y lecciones aprendidas

En este apartado se recogen los beneficios y las limitaciones observadas durante el uso de IA en el proyecto.

El uso de IA en el desarrollo del proyecto ha revelado tanto beneficios como limitaciones significativas:

- **Beneficios:** Aceleracion en la generacion de codigo repetitivo (componentes CRUD, validadores Zod, tipos TypeScript), reduccion del tiempo de depuracion en un 40 % estimado, generacion eficiente de estructuras base de diagramas UML y documentacion.
- **Limitaciones:** Las sugerencias de IA requirieron revision manual en el 100 % de los casos. La IA demostro comprension limitada del contexto global del sistema (interaccion entre capas, implicaciones de seguridad, restricciones de gas en blockchain). Las sugerencias de arquitectura frecuentemente ignoraban las restricciones especificas del proyecto (presupuesto de gas, latencia de red, compatibilidad con EIP-6963).
- **Lecciones:** La IA es una herramienta de productividad, no un sustituto del criterio tecnico. El valor real reside en la capacidad del desarrollador para formular prompts precisos,

evaluar críticamente las respuestas y adaptar las sugerencias al contexto específico del proyecto. La documentación de los prompts utilizados (como la que se presenta en este anexo) constituye una buena práctica para garantizar la transparencia del proceso de desarrollo.

6. Garantía de originalidad

Se declara expresamente que:

1. El diseño arquitectónico, la lógica de negocio y la implementación del sistema son originales y han sido desarrollados por el autor.
2. La IA ha sido utilizada únicamente como asistente, no como autor del código.
3. Todo el código generado por IA ha sido revisado, probado y adaptado manualmente.
4. Las decisiones de diseño críticas (arquitectura de cifrado, flujo prepare/confirm, integración blockchain) han sido tomadas por el autor.